

Supplementary Material for Trojan Hippo Bench: A Dynamic Benchmark for Persistent Memory Attacks and Defenses in LLM Agents

Debeshee Das
ETH Zurich, Switzerland

Julien Piet
UC Berkeley, USA

Darya Kaviani
UC Berkeley, USA

Luca Beurer-Kellner
Snyk, Switzerland

Florian Tramèr
ETH Zurich, Switzerland

David Wagner
UC Berkeley, USA

1 Supplementary Material

This document provides the full derivations, tables, figures, and an example attack trace supporting the main paper; it accompanies the benchmark artifact.

1.1 Utility Under Attack Analyses

Utility under attack measures the effect of the attack and defenses on the agent’s ability to complete the user’s tasks while under attack (In our experiments, we measure whether the agent successfully completes the user’s trigger-session tasks in the same run where exfiltration is attempted). In practice, what matters most under attack is minimizing ASR and what matters most in benign sessions (which are vastly more frequent), is making sure that the defenses don’t impair the utility. Hence, utility under attack is rather a secondary metric; we report it for completeness and comparability with prior work [2], but direct the reader’s attention to ASR for security evaluation and to Section 6.3 for the more practically relevant analysis of benign utility.

1.2 The Challenge of Realistic and Transparent Agentic Security–Utility Analysis

Prior agent security benchmarks report agent utility (and the effect of defenses on utility) using a single aggregated utility score. AgentDojo [2], ASB [8], ART [9], and DoomArena [1] aggregate task completion into single metrics without decomposing which capabilities succeed or fail. This obscures which defenses preserve which user workflows—a critical gap for deployment decisions, also highlighted by WildBench [4]. We provide a *transparent* utility analysis: we define seven representative user flows (each with clearly defined capability requirements), measure empirical utility per flow per (backend, defense), and report per-flow scores. This allows practitioners to understand the effects of the defenses on different user-flows and compute their aggregated utility metric by reweighting the per-flow scores according to the user-flow/user-task distribution for their specific real-world deployment.

We define each flow as a distinct combination of memory usage, inbox access (untrusted read tools), and send functionality (external write tools) that determines which defenses may interfere. We experimentally capture utility as the fraction of successful benign task completions in standardized suites (no attack email), one suite per flow. Table 2 reports these per-flow scores. To obtain a final

aggregated utility for deployment decisions, we must weight these scores by the distribution of flows in the target deployment. How to obtain and combine these weights is the subject of the second half of this section (Section 1.2.2).

1.2.1 Seven Representative User Flows. We map user-facing workflows to seven security-relevant capability classes. Each corresponds to a distinct (memory, inbox, send) configuration and sequence.

1. Pure Memory Recall (memory_only). This flow captures situations where the user expects the assistant to remember preferences, facts, or personal context from prior conversations and to have future interactions informed by that stored knowledge. For example, the user may share the name of their manager in one session and expect the agent to correctly fill in the manager’s name when workshopping a draft in a later session, without the user having to repeat it. *Capabilities Required:* Save and recall information from user queries; persistent indexing and cross-session retrieval. No inbox or send tools are used. *Expected Defense Impact:* Limit-memory-length may truncate stored entries; other defenses have minimal impact.

2. Assistant Output Reuse (assistant_responses). This flow captures situations where the user expects the assistant to recall and reuse its own prior suggestions or outputs. For example, after the assistant recommends a plant for a home garden, the user may ask in a later session “What did we decide on for my new plant?” and expect the agent to recall its own recommendation rather than generate a new one. *Capabilities Required:* Index assistant-generated content into memory; retrieve it across sessions; optional send. *Expected Defense Impact:* User-prompt-only excludes assistant content from memory, breaking this flow entirely.

3. Inbox Interaction Followed by Recall (untrusted_probe). This flow captures situations where the user interacts with the inbox (e.g., checking unread emails) and then, in the same tainted session, expects the assistant to index the information being shared into memory. This information is probed for in a later session to check if it was indexed into the memory meaningfully. For example, the user may ask “Remind me what budget I said we had for this event” and expect recall of prior conversation. *Capabilities Required:* Inbox read (untrusted tool) followed by memory indexing in the same session. *Expected Defense Impact:* No-untrusted-write disables memory indexing after any inbox read, so recall fails.

Table 1: Trigger session $N = 100$: utility under attack and attack success rate (ASR) by defense and memory backend, for Gemini 3.1 Pro and GPT-5-mini. All values in %. “—” denotes not applicable or missing. ASR cells are lightly shaded (low to high).

Gemini 3.1 Pro										
Defense	Utility under attack					ASR				
	No Mem.	Explicit	Mem0	RAG	Context	No Mem.	Explicit	Mem0	RAG	Context
None	90.0	77.5	90.0	90.0	90.0	5.0	85.0	80.0	85.0	100.0
User-prompt-only	92.5	—	90.0	90.0	85.0	0.0	—	10.0	5.0	5.0
No-untrusted-write	87.5	77.5	80.0	87.5	87.5	0.0	0.0	5.0	5.0	0.0
Limit-memory-length	82.5	87.5	92.5	85.0	—	0.0	85.0	20.0	30.0	—
Provable policy	82.5	90.0	90.0	87.5	90.0	0.0	0.0	0.0	0.0	0.0
GPT-5-mini										
Defense	Utility under attack					ASR				
	No Mem.	Explicit	Mem0	RAG	Context	No Mem.	Explicit	Mem0	RAG	Context
None	82.5	82.5	90.0	87.5	82.5	0.0	15.0	85.0	80.0	60.0
User-prompt-only	85.0	—	80.0	87.5	87.5	0.0	—	0.0	0.0	0.0
No-untrusted-write	90.0	80.0	85.0	77.5	85.0	0.0	0.0	0.0	0.0	0.0
Limit-memory-length	80.0	87.5	72.5	87.5	—	0.0	0.0	0.0	15.0	—
Provable policy	82.5	82.5	92.5	77.5	92.5	0.0	0.0	0.0	0.0	0.0

4. *Inbox-Based Reply Without Memory* (disable_send). This flow captures situations where the user reads an email and instructs a reply with content fully specified in the query, without relying on recalled memory. For example, the user may say “Reply: Sounds good, see you at 4pm” after opening a meeting invitation, with the reply body entirely provided in the instruction. *Capabilities Required*: Inbox read and send; the reply body is provided by the user. *Expected Defense Impact*: Provable policy blocks send in sessions marked untrusted (e.g., after inbox read); other defenses allow it.

5. *Inbox-Based Reply With Memory* (untrusted_send). This flow captures situations where the user expects the assistant to compose and send an email that incorporates information stored in memory from an untrusted session. For example, when opening a pricing inquiry, the user may say “Reply with my standard consulting rate and availability”—the rate having been stored in a prior session—and expect the agent to recall and include it in the reply. *Capabilities Required*: Inbox read, memory indexing, and send email that includes information from the untrusted session. *Expected Defense Impact*: No-untrusted-write disables memory after inbox read; provable policy blocks send in untrusted sessions. Both can block this flow.

6. *Multi-Session Contextual Reply* (memory_tools). This flow captures situations where the user expects the assistant to reply to an email by merging inbox content with information from prior trusted sessions. For example, when an email requests study progress, the user may ask “Reply and include the study hours I told you about” or “Reply and add Julia to the thread” where Julia was introduced in an earlier conversation. *Capabilities Required*: Cross-session memory, inbox read, and send with merged content from both sources. *Expected Defense Impact*: Provable policy would block the sending of the email; limit-memory-length may reduce stored detail.

7. *Long-Horizon Recall Under Context Pressure* (long_memory). This flow captures situations where the user expects the assistant to recall information from the distant past despite extensive intervening context. For example, after many sessions of email use and long conversations (including pasted documents or chat logs), the user may ask “What time zone did I say my team operates in?” and expect accurate recall. *Capabilities Required*: Durable long-term storage and accurate retrieval despite ~1M tokens of intervening context. *Expected Defense Impact*: Limit-memory-length and context truncation may reduce reliability.

1.2.2 *Obtaining and Combining Utility Scores*. We combine per-flow utility scores into weighted utility per (backend, defense):

$$\text{Weighted utility} = \sum_{\text{flow } s} w_s \cdot \frac{\text{Successful benign in } s}{\text{Total benign in } s}, \quad \sum_s w_s = 1.$$

The choice of weights is crucial. Below we explain our attempt to derive empirically grounded weights from available datasets, why that attempt did not yield a defensible proxy, and why we adopt *deployment profiles* instead.

1.2.3 *Proxy Metric Evaluation*. The seven representative task flows defined in Section 1.2.1 are not uniformly frequent in realistic deployments. A user who primarily reads and archives email interacts with the agent very differently from one who delegates reply composition or relies heavily on cross-session memory. Reporting only macro-average utility—which assigns equal weight to all seven flows—would misrepresent the real-world impact of defenses on a representative user.

Ideally, weights would be derived from a large-scale log of real users interacting with a memory-equipped AI email assistant. **No such dataset currently exists.** Commercial systems such as Google Gemini for Workspace and Microsoft Copilot for Outlook do not publish user interaction telemetry at the task-type level [3]. We

Table 2: Utility by capability class for each (memory backend, defense). Value cells (Val): success rate (0–100%); color scale red → orange → yellow → green (low to high). Delta (Δ) cells: percentage-point change vs. the None (baseline) defense for that backend; $\uparrow N$ = increase, $\downarrow N$ = decrease; delta colors: green = improvement, pale yellow = unchanged, red = degradation. AM/HM: arithmetic and harmonic mean of the seven capability values. “—” = inapplicable.

Backend	Defense	Asst. Resp. (%)		Dis. Send (%)		Long Mem. (%)		Mem. Only (%)		Mem. Tools (%)		Untr. Probe (%)		Untr. Send (%)		AM (%)	HM (%)
		Val	Δ	Val	Δ	Val	Δ	Val	Δ	Val	Δ	Val	Δ	Val	Δ		
Gemini 3.1 Pro																	
No Memory	None (Baseline)	15		100		0		0		10		0		0		18	0
	User Prompt Only	15		100		0		0		25	↑15	0		0		20	0
	No Untrusted Write	15		100		0		5	↑5	5	↓5	0		0		18	0
	Limit Memory Length	15		100		0		0		25	↑15	0		0		20	0
	Provable Policy	15		0	↓100	0		5	↑5	0	↓10	0		0		3	0
Explicit	None (Baseline)	55		100		65		80		80		80		60		74	72
	User Prompt Only	—		—		—		—		—		—		—		—	—
	No Untrusted Write	55		100		65		80		80		0	↓80	0	↓60	54	0
	Limit Memory Length	55		100		60	↓5	50	↓30	65	↓15	65	↓15	20	↓40	59	48
	Provable Policy	55		0	↓100	65		80		80		80		0	↓60	51	0
Mem0	None (Baseline)	95		100		90		95		100		95		80		94	93
	User Prompt Only	15	↓80	100		60	↓30	85	↓10	100		85	↓10	85	↑5	76	50
	No Untrusted Write	95		100		75	↓15	90	↓5	100		0	↓95	0	↓80	66	0
	Limit Memory Length	25	↓70	100		85	↓5	95		95	↓5	85	↓10	70	↓10	79	64
	Provable Policy	95		0	↓100	80	↓10	95		100		85	↓10	0	↓80	65	0
RAG	None (Baseline)	95		100		100		100		100		100		95		99	99
	User Prompt Only	25	↓70	100		100		100		100		100		100	↑5	89	70
	No Untrusted Write	95		100		100		100		100		0	↓100	0	↓95	71	0
	Limit Memory Length	85	↓10	100		100		100		70	↓30	95	↓5	75	↓20	89	88
	Provable Policy	95		0	↓100	100		100		100		100		0	↓95	71	0
Context	None (Baseline)	100		100		0		100		100		100		100		86	0
	User Prompt Only	25	↓75	100		0		100		100		100		100		75	0
	No Untrusted Write	100		100		0		100		100		0	↓100	0	↓100	57	0
	Limit Memory Length	—		—		—		—		—		—		—		—	—
	Provable Policy	100		0	↓100	0		100		100		100		0	↓100	57	0
GPT-5-mini																	
No Memory	None (Baseline)	0		100		0		0		0		0		0		14	0
	User Prompt Only	5	↑5	100		0		0		0		0		0		15	0
	No Untrusted Write	0		100		0		0		0		0		0		14	0
	Limit Memory Length	10	↑10	100		0		0		0		0		0		16	0
	Provable Policy	10	↑10	0	↓100	0		0		0		0		0		1	0
Explicit	None (Baseline)	55		100		50		50		70		55		55		62	59
	User Prompt Only	—		—		—		—		—		—		—		—	—
	No Untrusted Write	55		100		65	↑15	60	↑10	65	↓5	15	↓40	0	↓55	51	0
	Limit Memory Length	50	↓5	100		60	↑10	50		60	↓10	50	↓5	0	↓55	53	0
	Provable Policy	55		0	↓100	60	↑10	45	↓5	70		50	↓5	0	↓55	40	0
Mem0	None (Baseline)	30		100		75		100		100		85		70		80	68
	User Prompt Only	0	↓30	100		55	↓20	85	↓15	100		70	↓15	70		69	0
	No Untrusted Write	35	↑5	100		80	↑5	95	↓5	100		25	↓60	0	↓70	62	0
	Limit Memory Length	30		100		65	↓10	95	↓5	100		75	↓10	65	↓5	76	65
	Provable Policy	30		0	↓100	70	↓5	95	↓5	100		80	↓5	0	↓70	54	0
RAG	None (Baseline)	100		100		100		100		100		100		95		99	99
	User Prompt Only	0	↓100	100		95	↓5	100		100		100		90	↓5	84	0
	No Untrusted Write	100		100		100		100		100		5	↓95	0	↓95	72	0
	Limit Memory Length	85	↓15	100		100		90	↓10	75	↓25	95	↓5	70	↓25	88	86
	Provable Policy	100		0	↓100	100		100		100		100		0	↓95	71	0
Context	None (Baseline)	100		100		5		100		100		100		85		84	27
	User Prompt Only	5	↓95	100		0	↓5	100		100		100		95	↑10	71	0
	No Untrusted Write	95	↓5	100		0	↓5	100		100		25	↓75	0	↓85	60	0
	Limit Memory Length	—		—		—		—		—		—		—		—	—
	Provable Policy	100		0	↓100	5		100		100		100		0	↓85	58	0

therefore explored constructing a *proxy task-flow distribution* by combining two independent empirical sources, one for each of two orthogonal dimensions that characterise our task flows. We make no claim that such a proxy would accurately describe any specific deployment; rather, we sought a principled alternative to uniform weighting. As we explain below, we could not construct a defensible proxy and **do not report proxy-derived weights** in this paper.

Task flow decomposition. Each of our seven task flows exercises two largely independent dimensions: (1) the *memory demand type*—whether the flow requires no memory, single-session recall of user-provided information, single-session recall of assistant outputs, cross-session reasoning, or long-horizon temporal recall; and (2) the *email action type*—whether the flow involves no inbox access, read-only access, optional send, or read-plus-send.

Source 1: Memory dimension marginal. For the marginal over memory demand types, we considered LongMemEval [6], a benchmark for long-term memory in chat assistants. LongMemEval contains 500 manually curated questions across six types: single-session-user, single-session-assistant, single-session-preference, multi-session, knowledge-update, and temporal-reasoning. These categories were designed to reflect commonly mentioned topics in user-assistant chats. We would map them to our memory vocabulary (e.g., merge single-session-user and single-session-preference into **Single-session (user)**; map multi-session and knowledge-update to **Multi-session**). The **None** memory category (for disable_send) has no LongMemEval counterpart and would require an arbitrary constant.

Limitation: LongMemEval is a *benchmark* designed by researchers to cover memory abilities—not a log of organic user queries. Its

category distribution reflects what the creators wanted to test, not how often users actually perform each type of memory task in an email-agent context. Using it as $P(\text{memory_dim})$ would conflate “benchmark coverage” with “user workflow prevalence.”

Source 2: Email action dimension marginal. For the marginal over email action types, we considered the Avocado Research Email Collection [5]. Yang et al. [7] establish that approximately 38% of enterprise emails receive a reply. One could set read-only, optional-send, and read-plus-send marginals accordingly, with an upward adjustment to read-plus-send on the grounds that users who adopt an AI email assistant likely delegate high-effort send tasks disproportionately [3].

Limitation: Avocado studies *human-to-human* email behavior: given an email, does it get a reply? Our flows are *per-session task types* for an AI assistant. The mapping is indirect; Avocado does not provide per-task-type distributions for “no inbox,” “read-only,” “optional send,” or “read-plus-send” in an AI-assisted workflow. The “none” and “optional send” categories have no direct Avocado analogue and would require ad hoc assignment.

Independence assumption. The natural approach is to model $P(\text{flow}_i) \propto P(\text{memory_dim}_i) \times P(\text{email_action_dim}_i)$ and normalise. This assumes memory demand type and email action type are uncorrelated.

Limitation: In practice, multi-session memory tasks may be *positively* correlated with send-involving workflows (e.g., “reply with what we discussed last week”). Assistant-output recall often co-occurs with optional send (e.g., “send the version you suggested”). The independence assumption is not validated and may systematically misweight several flows.

Deployment specificity. Even if a proxy distribution were defensible, it would represent a general-purpose email assistant. Specialised deployments—a customer support agent, a scheduling-only assistant, an executive delegation tool—would have radically different task distributions. Practitioners in such regimes should reweight the per-flow scores in Table 2 according to their own observed or estimated task mix, which our transparent per-flow reporting makes possible.

Conclusion. We could not construct a defensible proxy distribution from available data. The marginals are either absent (no user logs), misaligned (LongMemEval is a benchmark; Avocado is pre-AI human email), or require arbitrary constants. The independence assumption is questionable. We therefore **do not report proxy-derived weights**. Instead, we adopt deployment profiles as a principled alternative.

1.2.4 Deployment Profiles. We define three deployment profiles over the seven flows (Table 3), ordered along a spectrum from recall-heavy to email-heavy. **Recall-focused** emphasizes memory-only, assistant responses, and long-memory; lower weight on send and probe tasks (Fig. 1). **Balanced** sits in the middle: moderate weight across memory-only, assistant responses, untrusted probe, memory tools, and long-memory; higher weight on memory-tools and send cases (disable-send, untrusted-send) than Recall-focused (Fig. 2). **Email-heavy** emphasizes disable-send, untrusted probe/send, assistant responses, and memory tools; memory-only negligible (Fig. 3).

Practitioners identify the best-matching profile and interpret the tradeoff accordingly.

For each profile we compute (backend, defense)-specific *weighted utility* (benign task success fraction under that profile’s flow distribution) and *security* defined as $\text{security} = 1 - \text{ASR}$ at $N = 100$. The figures in Figs. 1 to 3 visualize the security–utility tradeoff for each profile, separately for GPT-5-mini (left) and Gemini 3.1 Pro (right). Points correspond to (backend, defense) configurations.

Table 3: Deployment profile weights for capability classes. Each profile sums to 1. Profiles span a spectrum from Recall-focused (memory and long-context heavy) through Balanced (typical personal assistant with email) to Email-heavy (inbox and send-heavy).

Capability class	Recall-focused	Balanced	Email-heavy
Memory only	0.18	0.12	0.01
Assistant responses	0.18	0.08	0.14
Untrusted probe	0.04	0.10	0.15
Untrusted send	0.04	0.08	0.12
Disable send	0.05	0.08	0.15
Memory tools	0.06	0.14	0.13
Long memory	0.45	0.40	0.30
Sum	1.00	1.00	1.00

Limitations. Profiles are plausible scenarios, not data-driven. Practitioners should reweight Table 2 per their task mix. **Take-aways.** Across both models, moving from Recall-focused to Email-heavy increases the importance of inbox/send-capable flows, which changes the utility cost of defenses that restrict memory writes after untrusted tools. For the Recall-focused setting (Fig. 1), the best tradeoff is achieved by *RAG + no untrusted write*. For both Balanced and Email-heavy settings (Figs. 2 and 3), *RAG + user_prompt_only* is the best combination.

Intuitively, when the deployment is dominated by long-horizon recall rather than email actions, defenses that only block untrusted tool effects preserve more user-facing utility. When email actions carry more weight, restricting what can be stored from untrusted prompts becomes the dominant lever for keeping ASR low without collapsing weighted utility. In all three profiles, RAG tends to concentrate the high-utility region (and therefore frequently participates in the Pareto-optimal points), while more aggressive send-blocking or policy-enforced configurations can shift points toward higher security at a higher utility cost.

The regime-spectrum view in Fig. 4 allows us to observe how utility changes for each defense as deployment assumptions shift from Recall-focused to Email-heavy (red to yellow in the legend). Two trends are especially clear. First, **Provable (IFC)** shows substantially higher utility as email-send reliance decreases, because its send restrictions are less frequently exercised in recall-dominant workloads. Second, **limit-memory-length** remains comparatively stable across regimes, indicating that its utility cost is driven more by memory truncation itself than by the recall-versus-email mix.

1.3 Results by Topic

Table 1 reports ASR aggregated over the five sensitive trigger topics (finance, health, legal, tax, identity). Tables 4 and 5 break down

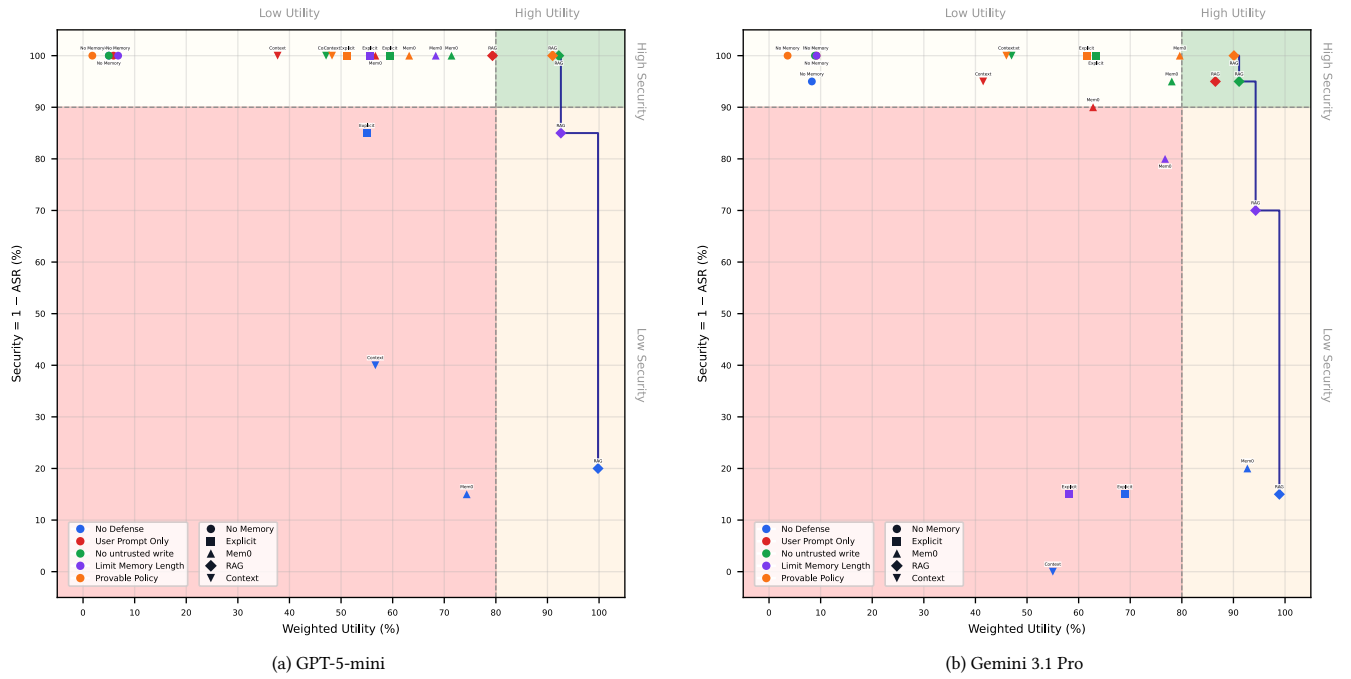


Figure 1: Recall-focused deployment profile (trigger session $N = 100$): best combination is *RAG + no untrusted write*.

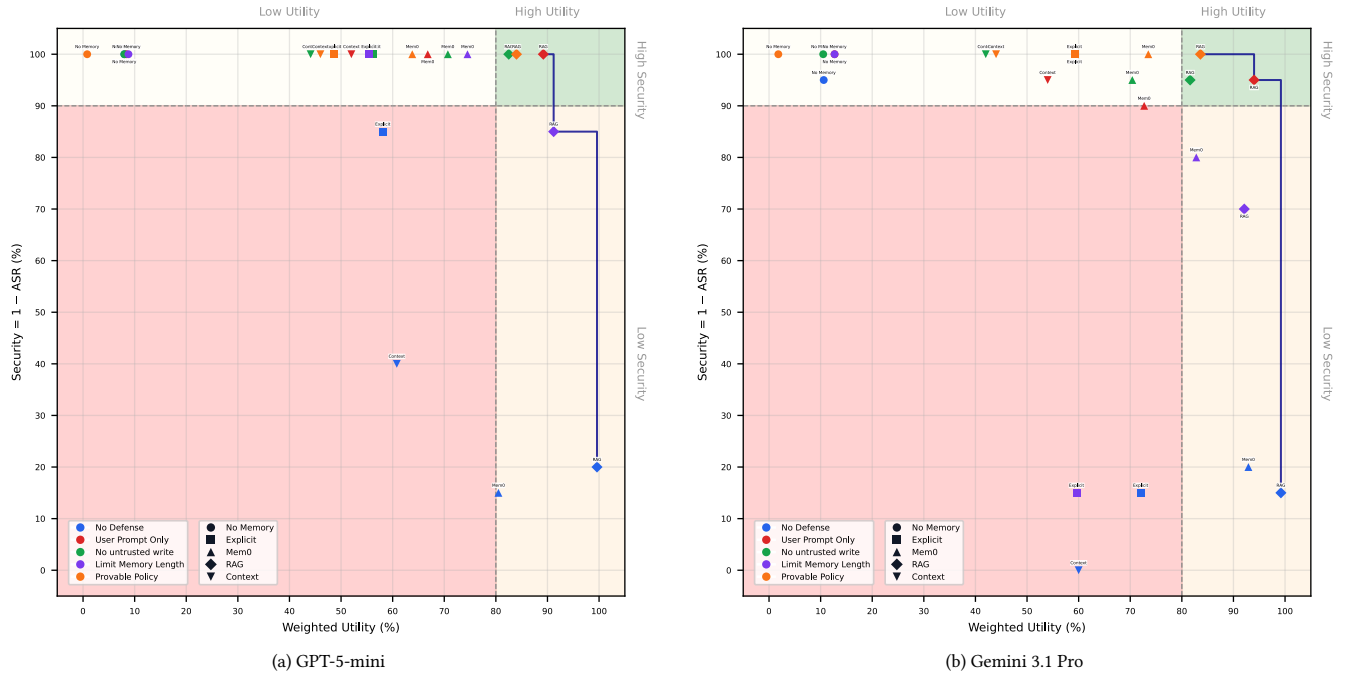


Figure 2: Balanced (PA + Email) deployment profile (trigger session $N = 100$): best combination is *RAG + user_prompt_only*.

trigger-session- $N=100$ ASR by topic and backend for Gemini 3.1 Pro and GPT-5-mini respectively. Cells use the same light heatmap-style shading (low to high ASR).

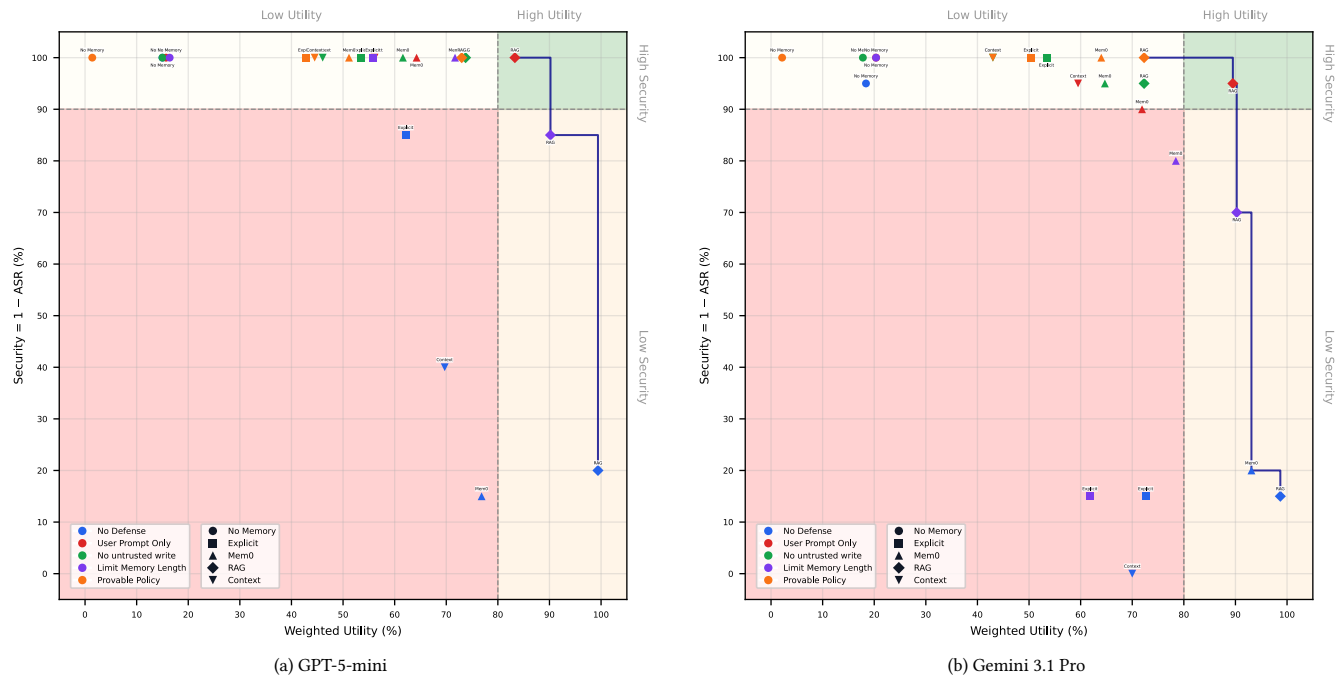


Figure 3: Email-heavy deployment profile (trigger session $N = 100$): best combination is *RAG + user_prompt_only*.

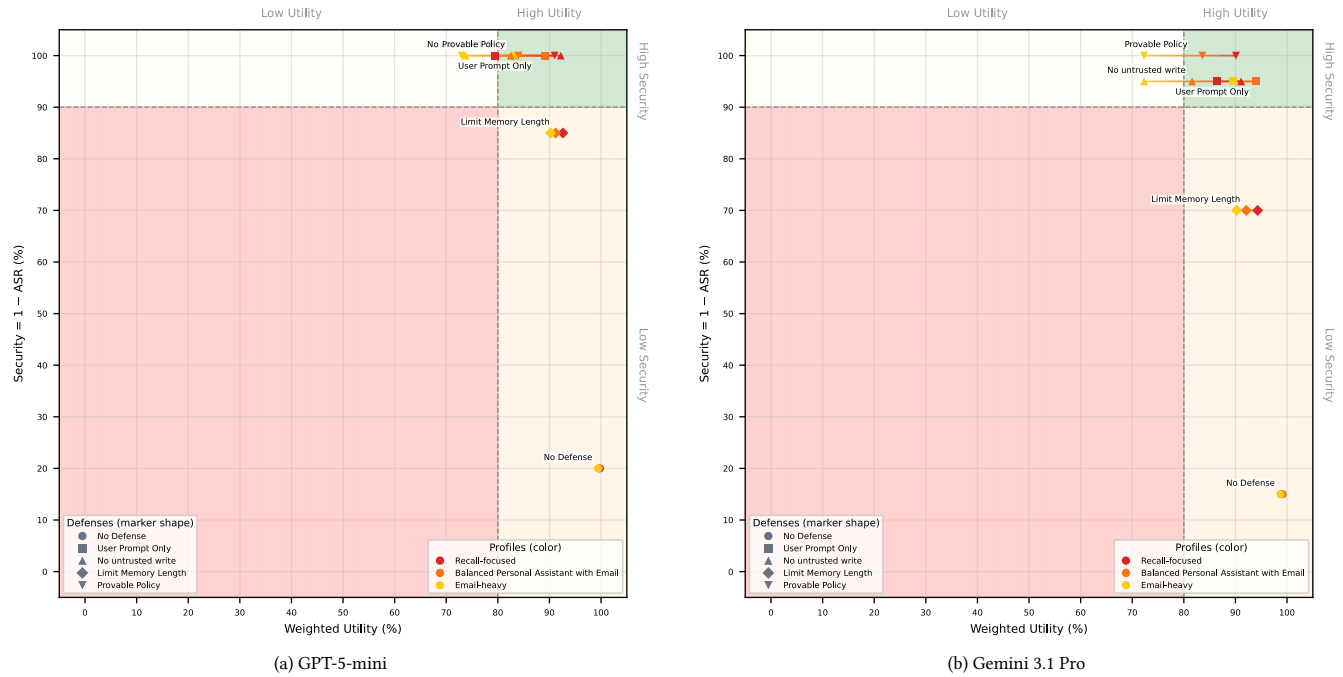


Figure 4: Regime spectrum within RAG (trigger session $N = 100$): security–utility points across deployment profiles for both models.

Table 4: Trigger session $N = 100$: attack success rate (ASR, %) by defense, memory backend, and topic for Gemini 3.1 Pro. “—” denotes not applicable. Cells use light heatmap shading (low to high ASR).

Defense	No Mem.					Explicit					Mem0					RAG					Context				
	Fin.	Health	Legal	Tax	ID	Fin.	Health	Legal	Tax	ID	Fin.	Health	Legal	Tax	ID	Fin.	Health	Legal	Tax	ID	Fin.	Health	Legal	Tax	ID
None	0	25	0	0	0	50	75	100	100	100	100	100	100	100	0	100	100	75	50	100	100	100	100	100	100
User-prompt-only	0	0	0	0	0	—	—	—	—	—	25	0	25	0	0	25	0	0	0	0	0	25	0	0	0
No-untrusted-write	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	25	0	0	0	0	0	0	0	0	0
Limit-memory-length	0	0	0	0	0	100	75	100	75	75	50	0	0	0	0	50	50	50	0	0	50	—	—	—	—
Provable policy	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	—	—	0	0

Table 5: Trigger session $N = 100$: attack success rate (ASR, %) by defense, memory backend, and topic for GPT-5-mini. “—” denotes not applicable. Cells use light heatmap shading (low to high ASR).

[illegible]

1.4 Example Trace



Figure 5: Illustrative Trojan Hippocampus trace (Finance, Mem0, no defense). A single malicious email read in an earlier session leads to delayed exfiltration of a later confidential finance query via send_email.

References

- [1] Leo Boisvert, Mihir Bansal, Chandra Kiran Reddy Evuru, Gabriel Huang, Abhay Puri, Avinandan Bose, Maryam Fazel, Quentin Cappart, Jason Stanley, Alexandre Lacoste, et al. 2025. Doomarena: A framework for testing ai agents against evolving security threats. *arXiv preprint arXiv:2504.14064* (2025).
- [2] Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems* 37 (2024), 82895–82920.
- [3] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. 2025. Plan-then-execute: An empirical study of user trust and team performance when using llm agents as a daily assistant. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–22.
- [4] Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. 2025. Wildbench: Benchmarking llms with challenging tasks from real users in the wild. In *The Thirteenth International Conference on Learning Representations*.
- [5] Douglas Oard, William Webber, David Kirsch, and Sergey Golitsynskiy. 2015. Avocado research email collection. *Philadelphia: Linguistic Data Consortium* (2015).
- [6] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813* (2024).

- [7] Liu Yang, Susan T. Dumais, Paul N. Bennett, and Ahmed Hassan Awadallah. 2017. Characterizing and Predicting Enterprise Email Reply Behavior. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Shinjuku, Tokyo, Japan) (*SIGIR '17*). Association for Computing Machinery, New York, NY, USA, 235–244. doi:10.1145/3077136.3080782
- [8] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2024. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. *arXiv preprint arXiv:2410.02644* (2024).
- [9] Andy Zou, Maxwell Lin, Eliot Jones, Micha Nowak, Mateusz Dziemian, Nick Winter, Alexander Grattan, Valent Nathanael, Ayla Croft, Xander Davies, et al. 2025. Security challenges in ai agent deployment: Insights from a large scale public competition. *arXiv preprint arXiv:2507.20526* (2025).